

Natural Language Processing

Свёрточные нейросети для классификации текстов

Фирсанова В.И.

НИУ ВШЭ

Бенчмарки для классификации текстов

Бенчмарки —

стандартизированные датасеты, метрики на лидербордах позволяют сравнивать модели и выявлять промышленные стандарты.

Основные бенчмарки:

- **GLUE** (9 задач, ~850k примеров) — анализ тональности, парафразы, импликации
- **SuperGLUE** — усложненная версия GLUE, появилась в связи с развитием NLP с архитектурой Transformer
- **SST-2/SST-5** (11k предложений, бинарная/5-классовая тональность) — The Stanford Sentiment Treebank
- **TREC** (6k вопросов, разметка вопросов по 6 типам)
- **IMDb** (50k отзывов, бинарная тональность)

Структура бенчмарка: GLUE

Задача	Описание	Метрика
CoLA	Лингвистическая приемлемость	Matthews corr.
SST-2	Анализ тональности	Accuracy
MRPC	Определение парафраз	F1/Accuracy
QQP	Являются ли вопросы парафразами	F1/Accuracy
MNLI	Определение логического следования	Accuracy
QNLI	Ответ на вопрос содержится в тексте	Accuracy
RTE	Распознавание текстовой импликации	Accuracy
WNLI	Определение кореферентности	Accuracy

Лидерборд SuperGLUE

SuperGLUE GLUE

Paper </> Code Tasks Leaderboard FAQ Diagnostics

Leaderboard Version: 2.0

	Rank	Name	Model	URL	Score	BoolQ	CB	COPA	MultiRC	ReCoRD	RTE	WiC	WSC	AX-b	AX-g
+	1	DeBERTa Team - Microsoft	DeBERTa / TuringNLRv4	🔗	90.3	90.4	95.7/97.6	98.4	88.2/63.7	94.5/94.1	93.2	77.5	95.9	66.7	93.3/93.8
+	2	Zirui Wang	T5 + Meena, Single Model (Meena Team - Google Brain)		90.2	91.3	95.8/97.6	97.4	88.3/63.0	94.2/93.5	92.7	77.9	95.9	66.5	88.8/89.9
	3	SuperGLUE Human Baselines	SuperGLUE Human Baselines	🔗	89.8	89.0	95.8/98.9	100.0	81.8/51.9	91.7/91.3	93.6	80.0	100.0	76.6	99.3/99.7
+	4	T5 Team - Google	T5	🔗	89.3	91.2	93.9/96.8	94.8	88.1/63.3	94.1/93.4	92.5	76.9	93.8	65.6	92.7/91.9
+	5	Huawei Noah's Ark Lab	NEZHA-Plus	🔗	86.7	87.8	94.4/96.0	93.6	84.6/55.1	90.1/89.6	89.1	74.6	93.2	58.0	87.1/74.4
+	6	Alibaba PAI&ICBU	PAI Albert		86.1	88.1	92.4/96.4	91.8	84.6/54.7	89.0/88.3	88.8	74.1	93.2	75.6	96.3/99.2
+	7	Tencent Jarvis Lab	RoBERTa (ensemble)		85.9	88.2	92.5/95.6	90.8	84.4/53.4	91.5/91.0	87.9	74.1	91.8	57.6	89.3/75.6
	8	Zhuyi Technology	RoBERTa-mtl-adv		85.7	87.1	92.4/95.6	91.2	85.1/54.3	91.7/91.3	88.1	72.1	91.8	58.5	91.0/78.1
	9	Facebook AI	RoBERTa	🔗	84.6	87.1	90.5/95.2	90.6	84.4/52.5	90.6/90.0	88.2	69.9	89.0	57.9	91.0/78.1
+	10	Infosys : DAWN : AI Research	RoBERTa-ICETS		84.0	86.1	93.2/95.2	91.2	80.1/45.9	89.9/89.3	87.9	71.3	89.0	35.2	93.8/68.8

LLM для классификации без дообучения

Zero-shot классификация

LLM получает описание классов в промпте и предсказывает метку без обучающих примеров.

Пример промпта:

"Классифицируй следующий текст по тональности. Классы: positive, negative, neutral. Текст: 'Фильм отличный, но концовка разочаровала.' Ответ:"

Few-shot классификация

LLM получает k размеченных примеров в промпте ($k = 3, 5, 10$) перед предсказанием.

Визуализация: zero-shot / few-shot классификация

Zero-Shot Learning

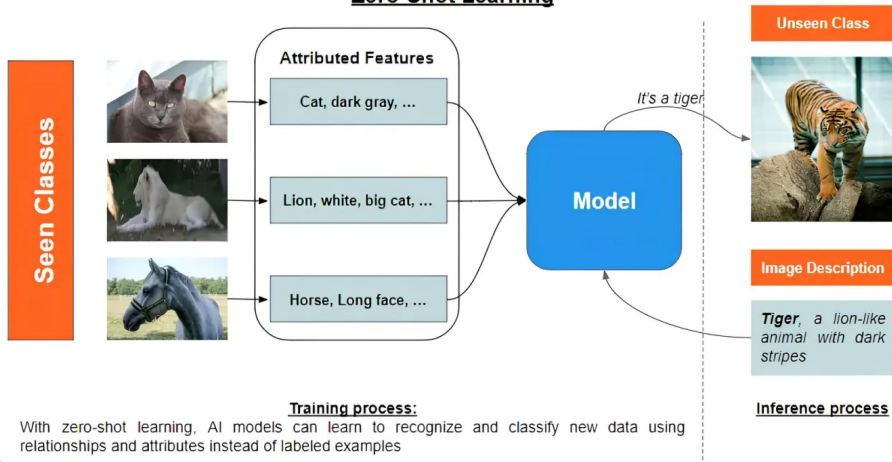


Рис.: <https://www.ultralytics.com/ru/blog/understanding-few-shot-zero-shot-and-transfer-learning>

Синтез данных для классификации через LLM

Метод

LLM генерирует текст и его метку на основе seed-набора или инструкции.

Пример промпта:

"Сгенерируй 10 коротких отзывов на книгу с меткой 'positive'. Пример формата: 'Увлекательный сюжет, читается на одном дыхании' → positive Выводи в JSON: 'text': str, 'label': str"

Параметры контроля: температура (темп. < 0.5 для детерминизма), top-p, стоп-слова

Проблемы датасетов, созданных LLM

Систематические ограничения

- **Низкое лексическое разнообразие** — LLM повторяет одни и те же паттерны
- **Смещение (bias)** — модель воспроизводит стереотипы из обучающих данных
- **Отсутствие редких классов** — LLM генерирует преимущественно частотные примеры
- **Контаминация** — LLM могла видеть тестовые данные при обучении

Эмпирическое правило: модели, обученные на синтетических данных, показывают на 5–15% ниже качество на реальных тестах (согласно работам Shumailov et al., 2023; Dohmatob et al., 2024).

<https://arxiv.org/abs/2305.17493>,

<https://arxiv.org/abs/2410.04840>

Синтез данных

Увеличение синтетических данных после определённого порога ведёт к **схлопыванию модели (model collapse)** — деградации качества на реальных данных.

Оценка качества классификации

Для оценки классификатора используются:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Когда что использовать:

- **Accuracy** — сбалансированные классы
- **F1 macro** — несбалансированные классы
- **ROC-AUC** — бинарная классификация с пороговым решением

LLM-as-a-Judge

Идея

Использование LLM для автоматической оценки качества классификации вместо ручной разметки.

Метрики, которые можно получить от LLM:

- Сравнение двух ответов ("Что лучше?")
- Оценка по шкале (например, 1-5)
- Определение ошибок: ложноположительные/ложноотрицательные

Известные бенчмарки:

- MT-Bench (Multi-Turn Benchmark) — оценка диалоговых моделей
- AlpacaEval — сравнение ответов моделей
- Chatbot Arena — парные сравнения с участием человека

LMArena

Battle Mode

hello world in c++

Option A
A is better

I'm going to create a single-page index.html that presents a clean C++ "Hello, World!" example with a copy-to-clipboard button and a small output preview.

Created index.html

Show More

Option B

I am going to create an interactive web application titled "CppCraft: The Ultimate C++ Hello World Studio". This single-page application will feature an interactive simulated C++ IDE with modern syntax highlighting, multiple C++ Hello World variations (from C++98 to C++23

Ask a follow up...

Add files Code

Inputs are processed by third-party AI and responses may be inaccurate.

Option A
A is better
Option B
B is better

Log in to view preview URL
Download

C++ Starter
Single File Example

Hello World in C++

This is the classic first C++ program. It includes the input/output library and prints **Hello, World!** to the console.

Copy code

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

1. Include

2. Main

3. Output

Проблемы датасетов, созданных LLM

Как проверять качество:

- Человеческая валидация на небольшой выборке
- Сравнение с традиционными датасетами
- Оценка через кросс-валидацию с разными LLM

Свёрточная нейросеть (CNN)

Основная идея

CNN извлекает локальные паттерны через **feature maps** (карты признаков)

Как это работает:

- ❶ Входные данные (текст/изображение) превращаются в набор признаков
- ❷ Остаётся только разбить признаки по классам

Важно

CNN создаёт карту признаков, где остаются только важные характеристики (углы, края, текстуры), а классификацию делает выходной слой.

Feature Map

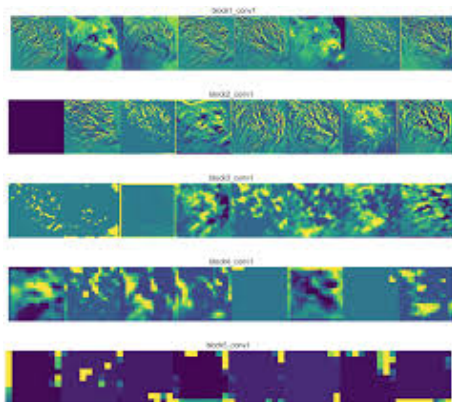


Рис.: <https://medium.com/@saba99/feature-map-35ba7e6c689e>

Входные данные: кодирование текста

Эмбединги слов

Текст кодируется с помощью эмбедингов (векторных представлений слов)

Схема преобразования:

Текст $\rightarrow$ Токены $\rightarrow$ Векторы (эмбединги) $\rightarrow$ Матрица

Для текстов используем Conv1D (одномерная свёртка), для изображений — Conv2D.

Conv1D, Conv2D, Conv3D

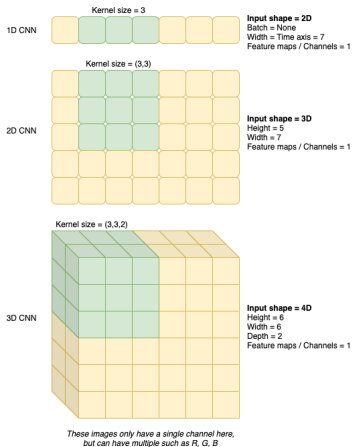


Рис.: <https://stackoverflow.com/questions/66220774/>

difference-between-the-input-shape-for-a-1d-cnn-2d-cnn-and-3d-cnn

Математическая операция свёртки

Определение

Свёртка — это поэлементное умножение ядра (kernel) на фрагмент входных данных с последующим суммированием результатов.

Формула для одного положения ядра:

$$\text{out} = \sum_{i=1}^k \sum_{j=1}^k \text{input}[x + i, y + j] \cdot \text{kernel}[i, j]$$

где k — размер ядра (kernel size).

Гиперпараметры фильтра:

- Размер ядра ($k \times k$)
- Шаг (stride) — на сколько пикселей сдвигается ядро
- Дополнение (padding) — добавление нулей по краям

Пример: свёртка изображения 5×5 с ядром 3×3

Вход (5×5):

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Ядро (kernel) 3×3 :

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Шаг (stride) = 1, без дополнения (padding = 'valid')

Визуализация: процесс свёртки

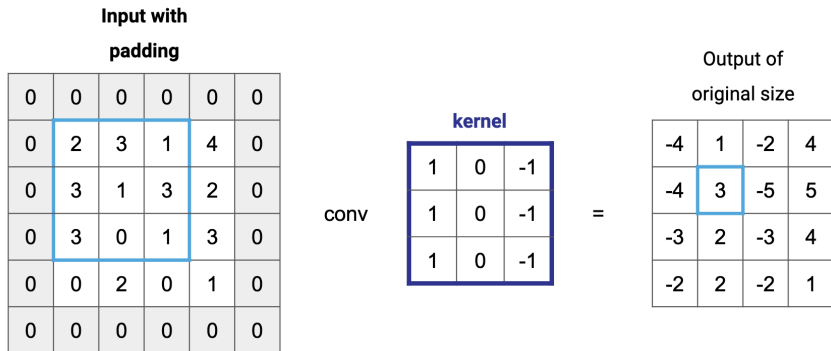


Рис.: <https://8thlight.com/insights/what-is-a-convolution-how-to-teach-machines-to-see-images>

Пример: вычисление первого значения

Позиция 1: верхний левый угол $(0, 0)$

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Поэлементное умножение:

$$\begin{aligned} &(1 \cdot 1) + (1 \cdot 0) + (1 \cdot 1) + \\ &(0 \cdot 0) + (1 \cdot 1) + (1 \cdot 0) + \\ &(0 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) \end{aligned}$$

Сумма: $1 + 0 + 1 + 0 + 1 + 0 + 0 + 0 + 1 = 4$

$$\text{out}[0, 0] = 4$$

Результат операции свёртки

Выходной размер (при $\text{stride}=1$, без padding):

$$\text{out_size} = \text{in_size} - k + 1 = 5 - 3 + 1 = 3$$

Выходная карта признаков (feature map) 3×3 :

$$\begin{bmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{bmatrix}$$

Ядро (kernel) — это веса, которые параметризуются (подбираются нейросетью в ходе обучения).

Карта признаков (Feature Map)

Определение

Feature map — результат применения свёрточного фильтра ко входным данным. Она используется непосредственно для выявления признаков классов.

Процесс:

Текст $\xrightarrow{\text{эмбединги}}$ Матрица $\xrightarrow{\text{свёртка}}$ Feature map

Алгоритм называется "скользящее окно" — окно (ядро) проходит по всей матрице, и операция повторяется множество раз.

На каждом слое значения в ядрах свёртки меняются, т.е. параметризуются.

Количество свёрточных фильтров

Иерархия признаков

Количество фильтров в CNN увеличивается для захвата более сложных иерархических признаков по мере уменьшения пространственных размеров.

Пример архитектуры:

- Ранние слои (~ 8 фильтров) — простые черты: края, линии
- Глубже ($16 \rightarrow 32$) — комбинации (углы, текстуры)

Несколько фильтров $\rightarrow$ мультиканальность (фильтр = канал)

Пример: 8 фильтров 5×5 скользят по 28×28 , $\text{stride}=1$, $\text{padding}=\text{'same'}$ $\rightarrow 24 \times 24 \times 8$

Операция пулинга (subsampling)

Цель пулинга

Уменьшение количества параметров в сети, снижение шума, сохранение наиболее важных признаков.

MaxPool 2×2 / stride = 2:

- Стандартный выбор: уменьшает размер в 2 раза
- Берёт максимальное значение в окне
- Извлекает контрастные признаки (подходит для классификации)

AvgPool: применяется для сглаживания признаков

Пример: из $24 \times 24 \times 8$ после MaxPool 2×2 /stride=2 $\rightarrow 12 \times 12 \times 8$

Пример: Max Pooling 2×2 , stride=2

Входная карта признаков 4×4 :

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 3 & 2 & 8 & 7 \\ 1 & 0 & 4 & 5 \end{bmatrix}$$

Результат MaxPool 2×2 :

$$\begin{bmatrix} 6 & 4 \\ 8 & 8 \end{bmatrix}$$

Формула MaxPool

$$\text{out}[x, y] = \max_{i, j \in \text{window}} \text{input}[x + i, y + j]$$

Flattening — подготовка для полносвязных слоёв

Проблема

Dense (полносвязный) слой не может обработать трёхмерный тензор, только одномерный вектор для матричного умножения.

После свёрток/пулинга карты признаков имеют форму (H, W, C) :

- $H \times W$ = размер карты (height $\times$ width)
- C = количество каналов (фильтров)

Flattening преобразует многомерные карты признаков в одномерный вектор:

$$(H, W, C) \xrightarrow{\text{flatten}} (H \times W \times C,)$$

Пример: $2 \times 2 \times 32 = 128 \rightarrow$ вектор длины 128

Визуализация: Flattening



Рис.: <https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-3-flattening>

Dense слои и финальная классификация

Что делает Dense слой

Dense слои используют одномерный вектор для нелинейных комбинаций и финальной классификации по классам.

Пример для датасета MNIST (цифры 0-9):

Финальный Dense(10, softmax) → вероятности классов (0-9 для цифр)

Важно

Классы определяются не по отдельным признакам, а всей архитектурой целиком — комбинацией всех слоёв, которая последовательно выделяет всё более абстрактные признаки, пока последний слой не сопоставит их итоговую комбинацию с конкретным классом. Ни один отдельный вес или нейрон за класс не отвечает.

Полная архитектура CNN для текста

Типичная последовательность:

- 1 Входные данные (текст) → эмбединги
- 2 Свёртка (Conv1D) — извлечение локальных паттернов
- 3 MaxPooling — уменьшение размерности
- 4 Повторение свёртки + пулинга
- 5 Flattening или GlobalMaxPooling
- 6 Dense слои для классификации
- 7 Выходной слой (softmax)

Визуализация: полная архитектура CNN

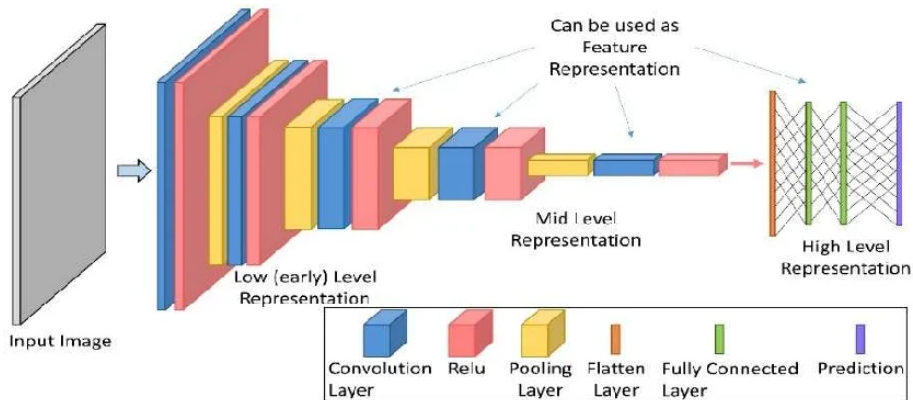


Рис.: https://www.researchgate.net/figure/General-CNN-architecture-Three-types-of-CNN-architectures-are-used-fig1_354032441

CNN на практике

<https://dsgiitr.github.io/mnist-cnn-demo/>

Что можно увидеть в демо

- Как фильтры извлекают признаки из цифр MNIST
- Как меняются feature maps после каждого слоя
- Визуализацию того, что "видят" разные фильтры

Что нужно запомнить

- ❶ **Свёртка:** поэлементное умножение ядра на фрагмент данных с суммированием

$$\text{out} = \sum_{i=1}^k \sum_{j=1}^k \text{input}[x + i, y + j] \cdot \text{kernel}[i, j]$$

- ❷ **Размер выхода свёртки:**

$$\text{out_size} = \text{in_size} - k + 1 \quad (\text{stride} = 1, \text{padding} = \text{'valid'})$$

- ❸ **MaxPool:** выбор максимального значения в окне

$$\text{out}[x, y] = \max_{i, j \in \text{window}} \text{input}[x + i, y + j]$$

- ❹ **Flattening:** преобразование $(H, W, C) \rightarrow (H \times W \times C,)$

- ❺ Количество фильтров растёт с глубиной: $8 \rightarrow 16 \rightarrow 32$